# Interp 2 : Written Assignment

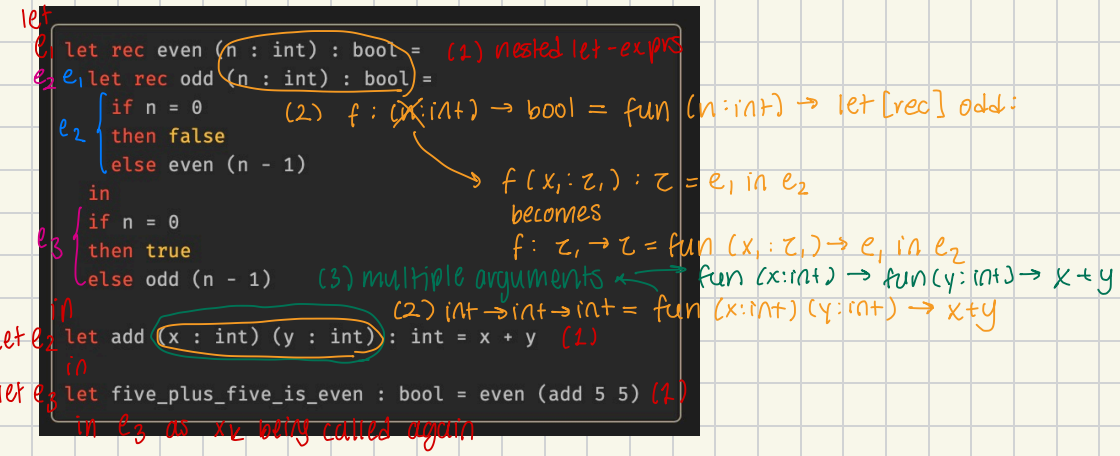
Desugaring - removing "syntactic sugar" / syntactic conveniences

(1) multiple top-level let defs are actually nested let exprs

(2) let-exprs with arguments are actually let-exprs with no arguments whose values are anonymous functions

(3) anynomous funcs with more than one argument are actually curry-ed collection of single-argument anonymous functions

1.) Desugaring:




Solution:

```
let [rec] even : int → bool =
    fun (n:int) →
      let [rec] odd : int → bool =
        fun (n:int) →
          if n=0
          then true
          else even (n-1)
      in
      if n=0
      then true
      else odd (n-1)
in
let add : int → int → int =
   fun (x:int) → fun (y:int) → x+y
in
 let five_plus_five_is_even : bool =
    even (add 5 5)
 in
 five_plus_five_is_even
```

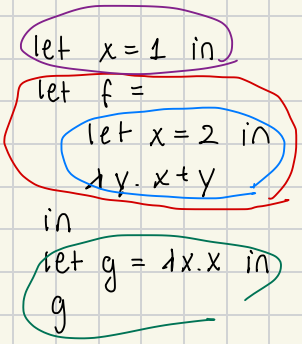
2.) closures

```
let x : int = 1 in
let f : int -> int =
  let x : int = 2 in
  fun (y : int) -> x + y
in
let g : int -> int = fun (x : int) -> x in
g
```

outermost let #1 → first env

let # 2

let # 3




$\mathcal{E} = \{x \mapsto 1\}$

$$\frac{\langle \mathcal{E}, 2 \rangle \Downarrow 2 \quad \langle \mathcal{E}\{x \mapsto 2\}, \lambda y.x+y \rangle \Downarrow (\{x \mapsto 2\}, \lambda y.x+y)}{\langle \{x \mapsto 1\}, \text{let } x=2 \text{ in } \lambda y.x+y \rangle \Downarrow (\{x \mapsto 2\}, \lambda y.x+y)}\ (\text{letEval})$$

$v_1$

(funEval) (varEval)

$$\frac{\langle \mathcal{E}, \lambda x.x \rangle \Downarrow (\mathcal{E}, \lambda x.x) \quad \langle \mathcal{E}[g \mapsto (\mathcal{E}, \lambda x.x)], g \rangle \Downarrow (\mathcal{E}, \lambda x.x)}{\{x \mapsto 1, f \mapsto (\{x \mapsto 2\}, \lambda y.x+y)\}, \langle \text{next\_expr} \rangle \Downarrow (\mathcal{E}, \lambda x.x)}\ (\text{letEval})$$

$$\frac{}{\langle \{x \mapsto 1\} \quad \text{let } f = \underbrace{\text{let } x=2 \text{ in } \lambda y.x+y}_{e_1} \text{ in } \langle \text{next\_expr} \rangle \Downarrow (\mathcal{E}, \lambda x.x)}$$

$x =$ in $e_2$

$\langle \text{next\_expr} \rangle = \text{let } g = \lambda x.x \text{ in } g$

$x = e_1 \text{ in } e_2$

$\mathcal{E} = \{x \mapsto 1, f \mapsto (\{x \mapsto 2\}, \lambda y.x+y)\}$

Answer:

$$\langle \{x \mapsto 1, f \mapsto (\{x \mapsto 2\}, \lambda y.x+y)\}, \lambda x.x \rangle$$

Semantic Derivations:

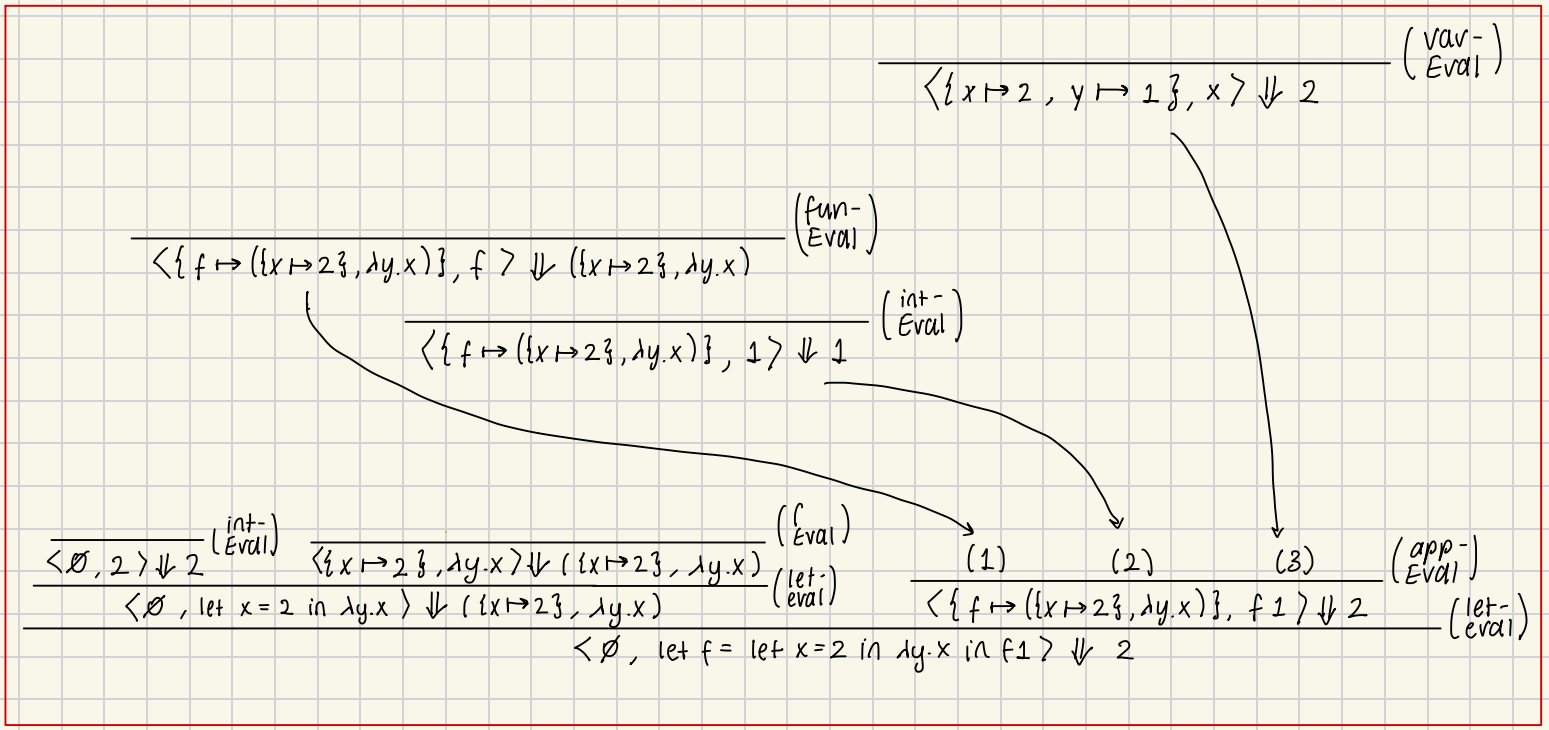
1.) let f : int → int =
    let x : int = 2 in
    fun (y : int) → x
    in f 1

$e_1$ = let f = (let x = 2 in λy.x) in f 1
(x; $e_1$ = let x = 2 in λy.x; $e_2$ = f 1; nested let-expr; application: $e_1$ $e_2$)

= let f = λy.2 in f 1

= let f = 2 in f 1

Hence, $e_1 \Downarrow 2$ and $v = 2$




2.) $e_2$ = let f = (let x = 2 in fun y → x) in f 1
(x = $e_1$ in $e_2$; x = $e_1$; in $e_2$)

= let f = fun y → 2 in f 1

= let f = 2 in f 1

Hence, $e_2 \Downarrow 2$ and $v = 2$

side conditions are only written for me to be able to correctly track substitutions.